

CUET PG (SCQP09) PRACTICE QUESTION BANK

Computer Science & Information Technology | Subject: Data Structures

TOPIC: QUEUES (MEDIUM LEVEL)

30 MCQs

Instructions:

- Each question carries 4 marks. For each incorrect response, 1 mark will be deducted (-1).
- Select the single best answer choice for each multiple-choice question.
- Answer Key is provided on the final page of this question sheet.

Q1. A circular queue of size $N = 6$ is stored in an array indexed 0 to 5. Currently, $FRONT = 3$ and $REAR = 1$. What is the total number of elements present in the queue?

- (A) 2 (B) 4
(C) 5 (D) 3

Q2. In a queue implemented using two stacks $S1$ and $S2$, to make Dequeue operation efficient ($O(1)$ amortized), elements are enqueued into $S1$. When Dequeue is called and $S2$ is empty, what operation is performed?

- (A) Return error underflow immediately (B) Pop all elements from $S1$ and push them into $S2$, then pop from $S2$
(C) Pop top element from $S1$ and return it directly (D) Clear $S1$ and reset all pointers

Q3. Statement I: In a Queue implemented using a Singly Linked List with head and tail pointers, both Enqueue and Dequeue operations can be performed in $O(1)$ time.

Statement II: Deleting from the tail end of a Singly Linked List takes $O(1)$ time.

- (A) Both Statement I and Statement II are true (B) Both Statement I and Statement II are false
(C) Statement I is true but Statement II is false (D) Statement I is false but Statement II is true

Q4. An array of size 5 implements a circular queue. The following operations are performed sequentially on an empty queue: Enqueue(10), Enqueue(20), Enqueue(30), Dequeue(), Dequeue(), Enqueue(40), Enqueue(50), Enqueue(60). What are the values of $FRONT$ and $REAR$ (0-indexed)?

- (A) $FRONT = 2$, $REAR = 0$ (B) $FRONT = 2$, $REAR = 4$
(C) $FRONT = 0$, $REAR = 4$ (D) $FRONT = 3$, $REAR = 1$

Q5. Match List I with List II:

List I: (P) Priority Queue, (Q) Circular Queue, (R) Deque, (S) Breadth First Search

List II: (1) Insertion/Deletion at both ends, (2) Heap Data Structure, (3) Queue Data Structure, (4) Modulo Arithmetic

- (A) P-2, Q-4, R-1, S-3 (B) P-4, Q-2, R-1, S-3
(C) P-2, Q-1, R-4, S-3 (D) P-3, Q-4, R-1, S-2

Q6. In an input-restricted deque, which set of operations is PERMITTED?

- (A) Insertion at rear only; Deletion at both front and rear (B) Insertion at both ends; Deletion at front only
(C) Insertion at front only; Deletion at front only (D) Deletion at rear only; Insertion at both ends

Q7. Assertion (A): A linear queue implemented using an array suffers from false overflow.

Reason (R): Even when free space exists at the beginning of the array, $REAR == Capacity - 1$ prevents further insertions.

- (A) Both (A) and (R) are true and (R) is the correct explanation of (A) (B) Both (A) and (R) are true but (R) is NOT the correct explanation of (A)
(C) (A) is true but (R) is false (D) (A) is false but (R) is true

Q8. If a queue is implemented using a Circular Doubly Linked List with a single pointer 'head', what is the time complexity of Enqueue and Dequeue operations?

- (A) $O(1)$ for Enqueue and $O(N)$ for Dequeue (B) $O(1)$ for both Enqueue and Dequeue
(C) $O(N)$ for both Enqueue and Dequeue (D) $O(N)$ for Enqueue and $O(1)$ for Dequeue

Q9. Which sequence of operations on an initially empty queue Q will result in the elements standing in the order 30, 40 (with 30 at Front)?

- (A) Push(10), Push(20), Pop(), Push(30), Pop(), Push(40) (B) Enqueue(10), Enqueue(20), Dequeue(), Enqueue(30), Dequeue(), Enqueue(40)
(C) Enqueue(10), Enqueue(20), Enqueue(30), Dequeue(), Dequeue(), Enqueue(40) (D) Enqueue(30), Enqueue(40), Enqueue(10), Dequeue()

Q10. Suppose you are given a Queue Q containing elements [1, 2, 3, 4, 5] with 1 at the Front. You pop elements one by one from Q and push them onto a Stack S. Then you pop all elements from S and enqueue them back into Q. What is the final state of Q from Front to Rear?

- (A) [1, 2, 3, 4, 5] (B) [5, 4, 3, 2, 1]
- (C) [1, 3, 5, 2, 4] (D) [5, 1, 2, 3, 4]

Q11. Statement I: A priority queue stored as an unsorted array takes $O(1)$ time for insertion and $O(N)$ time for deleting the minimum element.

Statement II: A priority queue stored as a sorted array takes $O(N)$ time for insertion and $O(1)$ time for deleting the minimum element.

- (A) Both Statement I and Statement II are true (B) Both Statement I and Statement II are false
- (C) Statement I is true but Statement II is false (D) Statement I is false but Statement II is true

Q12. What is the worst-case time complexity to check if an element X is present in a standard queue of size N without destroying the queue?

- (A) $O(1)$ (B) $O(\log N)$
- (C) $O(N)$ (D) $O(N \log N)$

Q13. In a circular queue of size N, if we do NOT leave one position empty and use full space N, how can we differentiate between FULL and EMPTY conditions?

- (A) By keeping a separate 'count' variable or flag (B) It is mathematically impossible
- (C) By checking if $FRONT == 0$ (D) By setting $REAR = FRONT + 1$

Q14. Consider a Max-Priority Queue implemented using a Max-Heap with 7 elements: [90, 80, 70, 30, 10, 60, 50]. After one Extract-Max operation, what will be the root element?

- (A) 70 (B) 80
- (C) 60 (D) 50

Q15. To implement a stack using two queues Q1 and Q2 such that Push operation is $O(1)$, what is the time complexity of the Pop operation?

- (A) $O(1)$ (B) $O(N)$
- (C) $O(\log N)$ (D) $O(N^2)$

Q16. Assertion (A): Breadth First Search (BFS) uses a Queue, while Depth First Search (DFS) uses a Stack.

Reason (R): BFS explores nodes level by level in First-In-First-Out order, whereas DFS explores as deep as possible before backtracking in Last-In-First-Out order.

- (A) Both (A) and (R) are true and (R) is the correct explanation of (A) (B) Both (A) and (R) are true but (R) is NOT the correct explanation of (A)
- (C) (A) is true but (R) is false (D) (A) is false but (R) is true

Q17. In a Priority Queue implemented using a Min-Heap of size N, what is the time complexity to find the MAXIMUM element in the priority queue?

- (A) $O(1)$ (B) $O(\log N)$
- (C) $O(N)$ (D) $O(N \log N)$

Q18. What is the minimum number of Queues required to implement a Priority Queue?

- (A) 1 queue for each priority level (B) Exactly 2 queues
- (C) Exactly 3 queues (D) Priority queues cannot be implemented using standard queues

Q19. Statement I: In a Double-Ended Queue (Deque), we can implement both Stack and Queue ADTs efficiently. Statement II: A Deque can be implemented using a Doubly Linked List where insertFront, insertRear, deleteFront, and deleteRear all run in $O(1)$ time.

- (A) Both Statement I and Statement II are true
- (B) Both Statement I and Statement II are false
- (C) Statement I is true but Statement II is false
- (D) Statement I is false but Statement II is true

Q20. Given a circular queue of size 8 (indices 0 to 7). Initially FRONT = 0, REAR = 0 (queue has 1 element at index 0). Elements are enqueued until the queue is full (leaving 1 slot empty). What will be the value of REAR?

- (A) 6
- (B) 7
- (C) 0
- (D) 5

Q21. Consider the following sequence of operations on an empty Queue Q: Enqueue(5), Enqueue(10), Dequeue(), Enqueue(15), Enqueue(20), Dequeue(), Dequeue(). Which element remains in the Queue?

- (A) 5
- (B) 10
- (C) 15
- (D) 20

Q22. Match List I with List II:

List I: (P) Queue, (Q) Priority Queue, (R) Deque, (S) Circular Queue

List II: (1) Sliding Window Maximum problem, (2) Disk Scheduling (FCFS), (3) Round Robin CPU Scheduling, (4) Dijkstra's Shortest Path Algorithm

- (A) P-2, Q-4, R-1, S-3
- (B) P-4, Q-2, R-3, S-1
- (C) P-2, Q-1, R-4, S-3
- (D) P-3, Q-4, R-1, S-2

Q23. In a queue implemented using a Singly Linked List with head and tail pointers, if we execute Dequeue() on a queue containing only ONE element, what updates must occur?

- (A) Only head is updated to NULL
- (B) Only tail is updated to NULL
- (C) Both head and tail are updated to NULL
- (D) Neither head nor tail changes

Q24. Assertion (A): In a priority queue, insertion always takes $O(1)$ time regardless of implementation.

Reason (R): New elements can simply be appended at the end without maintaining heap order or sorting.

- (A) Both (A) and (R) are true and (R) is correct explanation
- (B) Both (A) and (R) are true but (R) is NOT correct explanation
- (C) Both (A) and (R) are false
- (D) (A) is true but (R) is false

Q25. Which of the following conditions correctly identifies that an array-based Circular Queue of size N (with 1 slot kept empty) is FULL?

- (A) $(\text{REAR} + 1) \% N == \text{FRONT}$
- (B) $\text{REAR} == \text{FRONT}$
- (C) $\text{REAR} + 1 == N$
- (D) $(\text{FRONT} + 1) \% N == \text{REAR}$

Q26. An output-restricted deque allows which of the following operations?

- (A) Insertion at both ends; Deletion at one end only
- (B) Deletion at both ends; Insertion at one end only
- (C) Insertion and Deletion at rear only
- (D) Insertion and Deletion at front only

Q27. How many stack pop operations are needed to reverse a queue of N elements using an auxiliary stack?

- (A) N
- (B) $2N$
- (C) $N/2$
- (D) N^2

Q28. Statement I: A queue can be efficiently implemented using a single circular linked list with a pointer pointing to the LAST node.

Statement II: With a pointer to the last node in a circular singly linked list, both front (last->next) and rear (last) can be accessed in O(1) time.

- (A) Both Statement I and Statement II are true
 (B) Both Statement I and Statement II are false
 (C) Statement I is true but Statement II is false
 (D) Statement I is false but Statement II is true

Q29. What is the space complexity of reversing a queue of size N using recursion?

- (A) O(1) auxiliary space
 (B) O(N) due to call stack memory
 (C) O(log N) stack space
 (D) O(N²) space

Q30. Given a queue Q = [10, 20, 30, 40, 50] (10 at Front). If we perform: Dequeue(), Dequeue(), Enqueue(60), Peek(), what value is returned by Peek()?

- (A) 10
 (B) 20
 (C) 30
 (D) 60

ANSWER KEY (Q1 - Q30)

Q.	Ans	Q.	Ans	Q.	Ans
1	B	11	A	21	D
2	B	12	C	22	A
3	C	13	A	23	C
4	A	14	B	24	C
5	A	15	B	25	A
6	A	16	A	26	A
7	A	17	C	27	A
8	B	18	A	28	A
9	C	19	A	29	B
10	B	20	A	30	C